TECHNICAL PROJECT REPORT

Arduino-Based Biosensor Signal Acquisition and Computational Analysis Prototype

A simulation-based study of calibration, noise injection, and digital filtering

SENSOR INPUT → ADC → FILTER
CALIBRATION → DISPLAY

Arduino Uno R3 | 16×2 LCD | Python data analysis

Prepared by: Udhay Chowdhury
Affiliation: Department of Electrical and Electronic Engineering, Chittagong University of Engineering & Technology, Bangladesh

Platform: Tinkercad Circuits and Google Colab
Report date: November 2026
Project type: Simulation-based prototype and computational analysis

This report documents a learning and portfolio project. It does not describe a validated glucose measurement device.

Contents

Abstract **iv**

Project Summary **iv**

1 Introduction **1**

1.1 Motivation . . . 1

1.2 Objectives . . . 1

1.3 Scope and non-goals . . . 1

2 System Overview **1**

2.1 Functional architecture . . . 2

2.2 Hardware and software components . . . 2

2.3 Circuit connections . . . 3

3 Measurement Model and Calibration **4**

3.1 Analog-to-digital conversion . . . 4

3.2 Illustrative calibration equation . . . 4

3.3 Calibration observations . . . 4

3.4 Linear regression . . . 5

4 Noise Modeling and Digital Filtering **5**

4.1 Python Gaussian-noise demonstration . . . 6

4.2 Software-injected noise in the Arduino simulation . . . 6

4.3 Moving-average filter . . . 6

4.4 Initial raw-versus-filtered result . . . 7

4.5 Comparison of filter-window sizes . . . 7

5 Results and Discussion **8**

5.1 Summary of reported results . . . 8

5.2 Interpretation . . . 8

5.3 Limitations . . . 8

5.4 What the project demonstrates . . . 9

6 Future Work **9**

7 Conclusion **10**

8 Portfolio Presentation **10**

A Arduino Source Code **10**

B Python Analysis Code **12**

B.1 Calibration and regression 12

B.2 Gaussian-noise demonstration 13

B.3 Arduino CSV analysis and window comparison 13

C References **14**

List of Tables

1 Components and their roles in the simulated prototype. 2
2 LCD-to-Arduino pin mapping used in the circuit. 3
3 Reported calibration observations. 5
4 Comparison of moving-average window sizes using the submitted dataset. 7
5 Summary of the main numerical results obtained during the project. 8

List of Figures

1 Functional architecture of the simulation-based signal acquisition and analysis prototype. 2
2 Overall Tinkercad circuit showing the Arduino Uno, sensor-emulation potentiometer, LCD, contrast potentiometer, and wiring. 3
3 Tinkercad simulation showing the LCD displaying the sensor voltage and glucose-equivalent output. 4
4 Python calibration plot showing the five simulated measurement points and fitted linear regression line. 5
5 Python-generated constant signal with Gaussian noise, plotted against the ideal reference voltage. 6
6 Arduino raw-versus-filtered ADC plot from the software noise-injection experiment. . . 7
7 Python comparison of raw signal and moving-average outputs for 3-, 5-, and 9-sample windows. 8

Abstract

This project presents a simulation-based prototype for acquiring, processing, and displaying an electrical signal representative of a biosensor response. An Arduino Uno R3 reads the wiper voltage of a potentiometer through analog input A0. The potentiometer is used as a controllable signal source; it does not chemically detect glucose. A hypothetical linear calibration maps the voltage interval from 0.5 V to 4.5 V onto a glucose-equivalent concentration interval from 0 mM to 10 mM. A 16×2 character LCD displays the calculated voltage and concentration-equivalent value.

The computational work includes a five-point calibration dataset, least-squares regression, synthetic noise generation, and moving-average filtering. The reported regression yielded a slope of 2.4988 mM/V, an intercept of −1.2472 mM, and a coefficient of determination reported as 1.000000. Because the concentration values were calculated from the same assumed linear mapping, this fit is a software consistency check rather than independent evidence of sensor linearity. In a separate Python noise simulation, the measured mean and standard deviation were 2.4980 V and 0.0464 V, respectively. A software-injected Arduino noise test showed a reduction in standard deviation after five-point moving-average filtering. A comparison of three filter windows showed reductions of 38.051%, 51.998%, and 64.319% for window sizes of 3, 5, and 9 samples, respectively.

The project demonstrates foundational skills in microcontroller-based analog acquisition, calibration mapping, serial data handling, Python-based analysis, and digital filtering. Its principal limitation is that the sensing mechanism is emulated rather than implemented using a biochemical or electrochemical transducer. The results therefore support a proof-of-concept signal-processing workflow, not a claim of glucose detection performance.

Keywords: biosensor-inspired system, Arduino Uno, analog-to-digital conversion, calibration, moving-average filter, signal processing, Python, Tinkercad.

Project Summary

Project objective	Build and analyze a low-cost, simulation-based electronic interface for a sensor-like analog signal.
Signal source	10 kΩ potentiometer connected to Arduino analog input A0.
Controller	Arduino Uno R3, using the nominal 10-bit ADC range of 0–1023.
Display	HD44780-compatible 16×2 LCD operated in 4-bit mode.
Calibration	Hypothetical linear mapping: $C = 2.5(V - 0.5)$, clipped to 0–10 mM.
Noise experiments	Python Gaussian-noise demonstration and Arduino software-injected integer noise.
Filtering	Five-sample moving average on Arduino; 3-, 5-, and 9-sample windows compared in Python.
Validation level	Functional simulation and computational consistency checks; no biochemical validation.

1 Introduction

Biosensors convert a biological recognition event into a measurable physical signal. In an electronic measurement chain, the transducer output may require conditioning, analog-to-digital conversion, digital processing, calibration, and display. These stages are relevant to electronic instrumentation and to research areas such as wearable sensing, bioelectronics, and organic electrochemical transistor (OECT)-based sensors.

The purpose of this project is to build a simplified, reproducible electronics-and-software workflow around a sensor-like voltage. A potentiometer is used to produce a controllable analog input. The Arduino reads this input, converts it into a digital value, applies a chosen calibration equation, and displays the resulting concentration-equivalent estimate. Python is used to inspect calibration data, fit a linear model, generate synthetic noise, and compare moving-average filters.

The project is intentionally scoped as a **biosensor-inspired signal acquisition prototype**. It does not contain an enzyme electrode, glucose oxidase, electrochemical cell, potentiostat, OECT, or any other glucose-sensitive material. The term "glucose-equivalent" is used throughout to distinguish the calculated display value from an actual glucose measurement.

1.1 Motivation

The project was selected to connect basic EEE knowledge with an intended research direction in biosensors and bioelectronics. It provides a practical exercise in the electronic signal chain that may accompany a future sensor platform. The implementation also creates a small dataset that can be examined using common scientific Python libraries.

1.2 Objectives

1. Construct a potentiometer-based analog input circuit in Tinkercad Circuits.
2. Read the input voltage using the Arduino ADC and transmit measurements through the serial interface.
3. Implement an illustrative linear calibration model that maps voltage to a glucose-equivalent value.
4. Display the voltage and calculated value on a 16×2 LCD.
5. Implement a five-sample moving-average filter on the microcontroller.
6. Generate and analyze synthetic noisy signals using Python.
7. Compare moving-average window sizes using standard deviation as a measure of signal variation.

1.3 Scope and non-goals

The scope covers circuit simulation, embedded programming, numerical mapping, basic statistical analysis, and visualization. It does not include fabrication of a sensing electrode, biochemical functionalization, electrochemical characterization, human or clinical testing, or comparison against a reference glucose analyzer. Therefore, the project does not establish sensitivity, selectivity, limit of detection, accuracy, repeatability, or clinical suitability of a glucose biosensor.

2 System Overview

2.1 Functional architecture

The implemented signal path is summarized in Fig. 1. The sensor potentiometer represents a variable transducer output. For the noise-injection experiment, a pseudo-random integer offset is added in software to the ADC reading before the moving-average filter. The filtered value is converted to voltage and then to the concentration-equivalent display value.

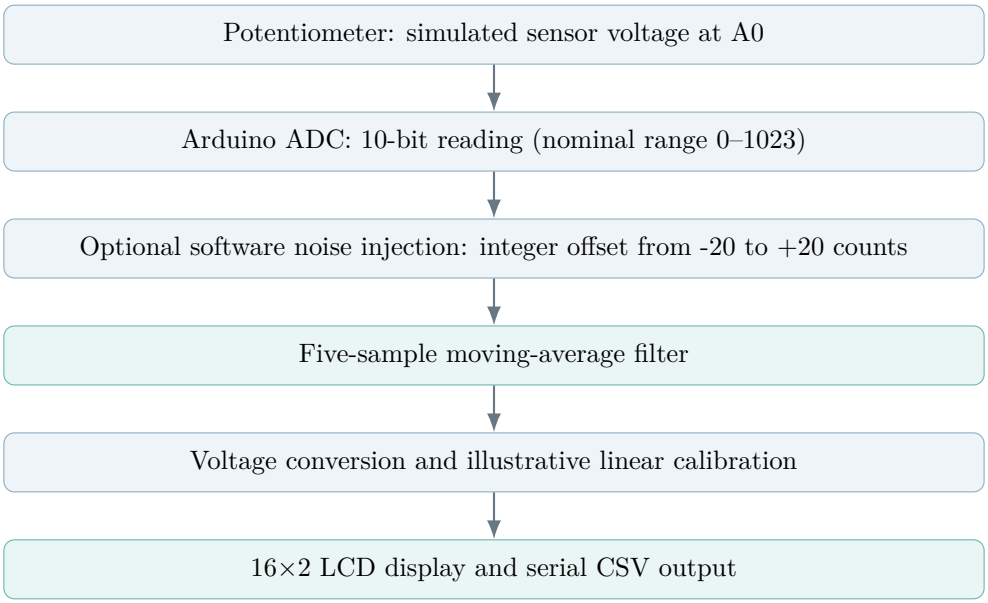


Figure 1: Functional architecture of the simulation-based signal acquisition and analysis prototype.

2.2 Hardware and software components

Table 1: Components and their roles in the simulated prototype.

Component	Configuration	Purpose
Arduino Uno R3	ATmega-based development board	Reads the analog input, performs calculations, and controls the LCD.
Sensor-emulation potentiometer	10 kΩ nominal	Generates a manually adjustable voltage representing a hypothetical sensor output.
Character LCD	16 columns × 2 rows	Displays filtered voltage and calculated glucose-equivalent concentration.
LCD contrast potentiometer	10 kΩ nominal	Adjusts the LCD contrast through its V0 pin.
Backlight resistor	220 Ω	Limits current through the simulated LCD backlight circuit.
Jumper wires	As required	Connects power, ground, analog input, and LCD control/data pins.
Tinkercad Circuits	Browser-based simulation	Provides the circuit environment used for the prototype.
Python environment	Google Colab; NumPy, pandas, Matplotlib	Performs regression, noise simulation, filtering comparison, and plotting.

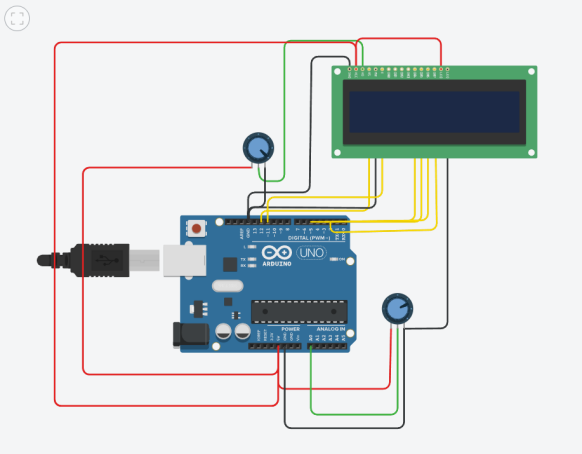


Figure 2: Overall Tinkercad circuit showing the Arduino Uno, sensor-emulation potentiometer, LCD, contrast potentiometer, and wiring.

2.3 Circuit connections

The sensor-emulation potentiometer is connected across the Arduino 5 V and ground rails. Its wiper (middle terminal) is connected to analog input A0. Rotating the potentiometer changes the voltage presented to A0.

The LCD uses 4-bit parallel communication. The RS and Enable signals are connected to digital pins 12 and 11, respectively. LCD data pins D4–D7 are connected to Arduino digital pins 5, 4, 3, and 2. The LCD read/write pin is tied to ground because the program writes to the display. The contrast potentiometer wiper is connected to LCD pin V0. The backlight anode is connected to 5 V through a 220 Ω resistor and the cathode is connected to ground. The Arduino LiquidCrystal library supports compatible character LCDs in 4-bit or 8-bit mode [3].

Table 2: LCD-to-Arduino pin mapping used in the circuit.

LCD pin	LCD signal	Connection
1	VSS	GND
2	VDD	5 V
3	V0	Wiper of contrast potentiometer
4	RS	Arduino D12
5	RW	GND
6	E	Arduino D11
7–10	D0–D3	Not connected (4-bit mode)
11	D4	Arduino D5
12	D5	Arduino D4
13	D6	Arduino D3
14	D7	Arduino D2
15	A / LED+	5 V through 220 Ω resistor
16	K / LED-	GND

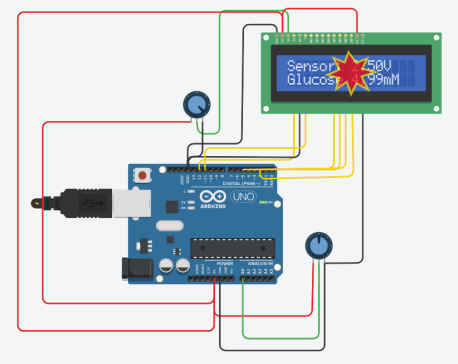


Figure 3: Tinkercad simulation showing the LCD displaying the sensor voltage and glucose-equivalent output.

3 Measurement Model and Calibration

3.1 Analog-to-digital conversion

The Arduino reads the wiper voltage using `analogRead(A0)`. For the Uno configuration used in this project, the ADC reading is treated as a 10-bit value from 0 to 1023. The nominal conversion used in the code is

$$V_{\text{in}} = \frac{D}{1023} V_{\text{ref}}, \tag{1}$$

where D is the ADC code and V_{ref} is assumed to be 5 V. This conversion follows the standard Arduino potentiometer/analog-input example [1]. The use of a nominal 5 V reference is an approximation; a physical implementation should measure or otherwise account for the actual reference voltage and ADC behavior.

3.2 Illustrative calibration equation

A hypothetical linear mapping was selected for demonstration. The lower calibration point is 0.5 V, assigned 0 mM; the upper point is 4.5 V, assigned 10 mM. Linear interpolation gives

$$C = C_{\min} + \frac{V - V_{\min}}{V_{\max} - V_{\min}}(C_{\max} - C_{\min}), \tag{2}$$

where C is a calculated glucose-equivalent concentration. Substituting the selected endpoints,

$$\boxed{C_{\text{eq}} = 2.5(V - 0.5) = 2.5V - 1.25} \qquad \text{for } 0.5 \le V \le 4.5. \tag{3}$$

The program clips values below zero to 0 mM and values above ten to 10 mM. This prevents extrapolated display values outside the chosen calibration interval. The clipping is a software boundary condition, not a physical property of a glucose sensor.

3.3 Calibration observations

Five voltage/concentration-equivalent pairs were recorded from the Arduino output. The concentration column is the output of the assumed calibration equation, not an independently measured glucose reference.

Table 3: Reported calibration observations.

Target voltage (V)	Reported voltage (V)	Reported C_{eq} (mM)	Comment
0.5	0.499	0.000	Lower limit applied
1.5	1.500	2.501	Within calibration interval
2.5	2.498	4.994	Within calibration interval
3.5	3.500	7.499	Within calibration interval
4.5	4.501	10.000	Upper limit applied

The voltage values are printed to a limited number of decimal places, while the concentration calculation uses the internal floating-point voltage. Small apparent discrepancies between a displayed voltage and its paired concentration-equivalent value can therefore result from rounding.

3.4 Linear regression

A least-squares line was fitted in Python to the five reported voltage and concentration-equivalent pairs. The reported fit was

$$C_{\text{eq,fit}} = 2.4988V - 1.2472, \tag{4}$$

with slope 2.4988 mM/V, intercept −1.2472 mM, and reported $R^2 = 1.000000$ (rounded). The fitted slope is close to the nominal equation slope of 2.5 mM/V.

Interpretation caveat. The concentration-equivalent values were generated from the same linear voltage-to-concentration equation that the regression attempts to recover. Consequently, an R^2 near one is expected by construction. It confirms that the Python regression reproduces the programmed mapping; it does not independently establish the linearity, accuracy, or sensitivity of a real glucose biosensor.

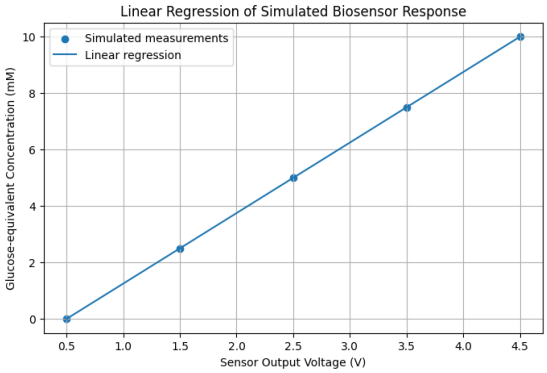


Figure 4: Python calibration plot showing the five simulated measurement points and fitted linear regression line.

4 Noise Modeling and Digital Filtering

4.1 Python Gaussian-noise demonstration

A separate Python experiment was used to understand the effect of random measurement variation. A constant ideal signal of 2.5 V was combined with normally distributed random values:

$$V_{\text{noisy}} = V_{\text{true}} + n, \qquad n \sim \mathcal{N}(0, \sigma^2), \tag{5}$$

where $V_{\text{true}} = 2.5\,\text{V}$, $\sigma = 0.05\,\text{V}$, and 200 samples were generated using a fixed random seed. The reported sample had a mean of 2.4980 V and a standard deviation of 0.0464 V. The difference between the observed and specified standard deviation is expected for a finite random sample.

This Gaussian-noise exercise was performed in Python and should not be confused with the later Arduino noise-injection experiment, which used a discrete uniform integer offset.

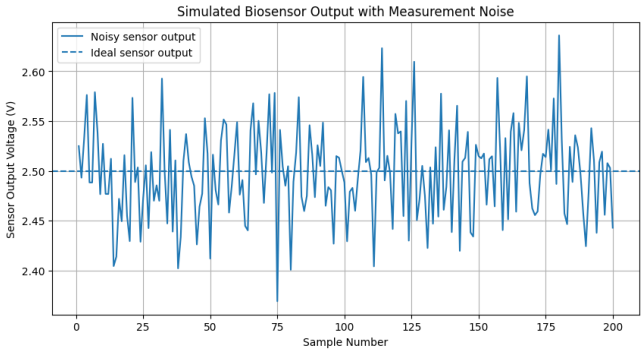


Figure 5: Python-generated constant signal with Gaussian noise, plotted against the ideal reference voltage.

4.2 Software-injected noise in the Arduino simulation

To test the embedded filter, a pseudo-random integer was added to the ADC reading:

$$D_{\text{noisy}} = \text{clip}(D_{\text{sensor}} + r, 0, 1023), \qquad r \sim \mathcal{U}\{-20, -19, \ldots, 20\}. \tag{6}$$

The code uses `random(-20, 21)`. In Arduino, the upper bound of the `random()` interval is exclusive, so this produces integer values from -20 through +20 [2]. The injected perturbation is therefore a discrete uniform software disturbance, not measured circuit noise and not Gaussian noise. The seed was initialized using an analog read from an unconnected A1 input in the simulation; this is a simple demonstration mechanism, not a robust entropy source for a deployed device.

4.3 Moving-average filter

A moving-average filter of length N computes the average of the current and previous $N - 1$ samples:

$$y[k] = \frac{1}{N} \sum_{i=0}^{N-1} x[k - i]. \tag{7}$$

For the Arduino implementation, $N = 5$. A circular buffer stores the latest five ADC values. The program subtracts the oldest value from a running sum, inserts the newest value, adds it to the sum, advances the buffer index, and divides the sum by five. This avoids recomputing the full sum at every loop iteration.

The LCD is updated every 200 ms in the noise-injection version, corresponding to a nominal update rate of 5 samples per second. With five samples in the averaging window, smoothing is accompanied by response delay. For an ideal uniformly sampled causal five-point moving average at 5 Hz, the nominal group delay is $(N - 1)/(2f_s) = 0.4$ s. Actual simulator scheduling and display updates may affect the observed settling behavior.

4.4 Initial raw-versus-filtered result

In one submitted Arduino noise-injection run, the raw ADC standard deviation was 12.987 counts and the filtered ADC standard deviation was 5.692 counts. The reported reduction was 56.17%. The submitted comparison plot showed the raw signal fluctuating more strongly than the filtered output.

$$\text{Variation reduction} = \left(1 - \frac{\sigma_{\text{filtered}}}{\sigma_{\text{raw}}}\right) \times 100\%. \tag{8}$$

This statistic describes the reduction in standard deviation in the analyzed sample. Because the input may contain slow changes as well as injected noise, it should be described as a reduction in signal variation rather than a complete measure of noise-removal performance. The initial 56.17% result used the sample selection from the first raw-versus-filtered comparison. The later window-size comparison trimmed the same number of edge samples for all three windows; its 5-sample result (51.998%) is the appropriate value for comparing window sizes within that analysis. The two percentages should not be pooled.

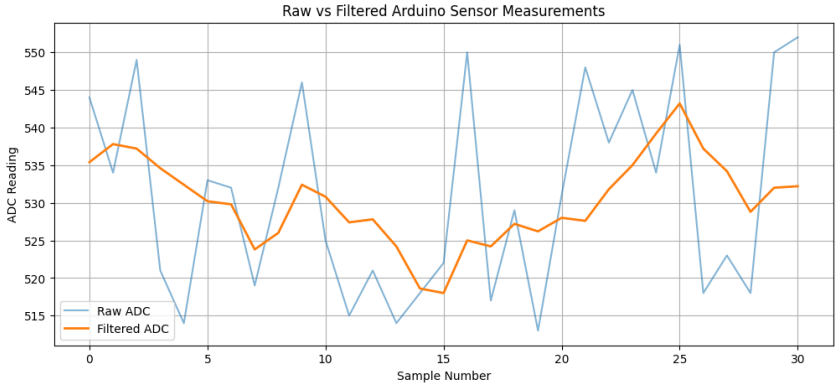


Figure 6: Arduino raw-versus-filtered ADC plot from the software noise-injection experiment.

4.5 Comparison of filter-window sizes

The Python analysis compared centered moving-average windows of 3, 5, and 9 samples. To make the comparison fair, the same interior sample positions were used for all windows by trimming the endpoints affected by the largest window. The reported results are summarized in Table 4.

Table 4: Comparison of moving-average window sizes using the submitted dataset.

Window (samples)	Filtered SD (ADC counts)	SD reduction (%)
3	7.633	38.051
5	5.915	51.998
9	4.396	64.319

The 9-sample window gave the lowest standard deviation, while the 3-sample window retained more of the short-term variation. The 5-sample window provided an intermediate result and was retained in the Arduino prototype as a compromise between smoothing and response speed. The selected window is not claimed to be optimal for an actual biosensor; choosing a window for a real sensing application would require a measured noise spectrum and an acceptable response-time requirement.

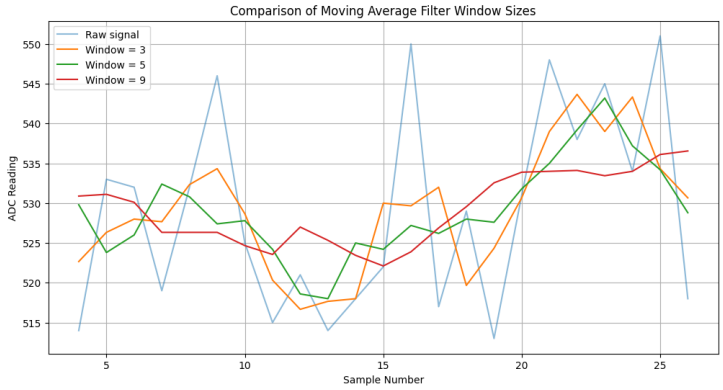


Figure 7: Python comparison of raw signal and moving-average outputs for 3-, 5-, and 9-sample windows.

5 Results and Discussion

5.1 Summary of reported results

Table 5: Summary of the main numerical results obtained during the project.

Analysis	Reported result
ADC conversion	Nominal 10-bit range: 0–1023, with a 5 V reference assumption.
Calibration mapping	$C_{\text{eq}} = 2.5(V - 0.5)$, clipped to 0–10 mM.
Linear regression	Slope = 2.4988 mM/V; intercept = -1.2472 mM; reported $R^2 = 1.000000$.
Python Gaussian-noise simulation	Mean = 2.4980 V; standard deviation = 0.0464 V for 200 samples.
Arduino software noise test	Raw ADC standard deviation = 12.987 counts; filtered = 5.692 counts; initial reported reduction = 56.17%.
Window-size comparison	3 samples: 38.051%; 5 samples: 51.998%; 9 samples: 64.319% reduction in standard deviation.

5.2 Interpretation

The circuit simulation demonstrated the basic operation of a microcontroller signal-acquisition chain: a variable analog voltage was read at A0, converted to a digital code, mapped to a voltage, transformed through a calibration equation, and shown on the LCD. Serial output provided a machine-readable route for collecting values in Python. The Python exercises demonstrated how to organize measurements, perform a linear fit, generate synthetic noise, visualize signal variation, and compare digital filters.

The Arduino filter comparison showed that a longer averaging window produced a lower standard deviation for the analyzed dataset. This is consistent with the behavior of a moving average applied to high-frequency random variation. However, the plotted signal was not perfectly stationary, and standard deviation alone cannot distinguish injected noise from genuine changes in the underlying input. In a real biosensor, over-smoothing could obscure fast analyte changes or transient events.

5.3 Limitations

The following limitations are central to interpreting the results:

1. **No biochemical recognition element:** the potentiometer does not respond to glucose or any other analyte.
2. **Illustrative calibration:** the voltage-to-concentration relationship was selected by the project designer and was not fitted to independently prepared glucose standards.
3. **No physical sensor measurements:** the circuit and the noise-injection workflow were simulated in Tinkercad. The noise added in the Arduino code was software-generated.
4. **No electrochemical front end:** the project does not measure sensor current, electrode potential, impedance, or OECT channel response.
5. **Limited dataset:** the submitted Arduino comparison plot contains a short run. Larger repeated datasets and multiple independent runs are needed for stronger statistical conclusions.
6. **Simplified voltage conversion:** the program assumes a nominal 5 V ADC reference. A practical circuit should account for reference accuracy, supply variation, offsets, and ADC quantization.
7. **Filtering trade-off:** lower standard deviation does not automatically mean higher accuracy. A moving average can introduce delay and attenuate rapid changes.

5.4 What the project demonstrates

The strongest defensible outcomes are evidence of practical learning in analog input acquisition, Arduino programming, LCD interfacing, serial data formatting, calibration implementation, Python regression, noise modeling, and moving-average filtering. The project is a useful stepping stone toward a laboratory sensor interface, but it should not be represented as a validated glucose biosensor or as experimental bioelectronics research.

6 Future Work

The prototype can be extended in stages to increase its research relevance and experimental rigor.

1. **Physical hardware implementation:** reproduce the simulated circuit on a breadboard, measure the actual 5 V reference, and verify ADC behavior using a calibrated multimeter or voltage source.
2. **Real signal acquisition:** replace the potentiometer with a documented sensor or an electrical test signal generated by a function generator. Record raw measurements without software-injected noise.
3. **Sensor calibration experiment:** prepare known analyte concentrations, acquire independent sensor responses, repeat each measurement, and estimate calibration parameters from the resulting experimental data.
4. **Electrochemical interface:** for an enzyme-based glucose sensor, investigate an appropriate electrode system and potentiostat/transimpedance front end. The required circuit depends on whether the sensor output is current, voltage, or impedance.
5. **OECT-oriented extension:** investigate how a chemical or ionic gate input changes OECT drain current, then design an acquisition stage suitable for the transistor's output characteristics. This would require an OECT device or a separately validated device model; it is not included in the present prototype.
6. **Improved signal processing:** compare moving average, median, and low-pass filters using measured data. Evaluate noise reduction together with rise time, settling time, and distortion of step changes.

7. **Larger datasets and reproducibility:** collect at least 100–200 samples per condition across repeated runs, preserve raw CSV files, document the sampling interval, and publish the analysis notebook with fixed software versions.

7 Conclusion

A simulation-based Arduino signal acquisition and computational analysis prototype was developed using a potentiometer as a controllable analog signal source. The Arduino Uno read the signal through A0, converted the ADC code into a nominal voltage, applied an illustrative linear mapping to a glucose-equivalent concentration, and displayed the output on a 16×2 LCD. Python was used to inspect the calibration mapping, perform regression, generate synthetic noise, and compare moving-average filter windows.

The five-point regression produced a slope close to the assumed calibration slope, as expected because the output concentration values were derived from that same linear equation. Software-injected noise experiments showed reduced signal variation after moving-average filtering. The window-size comparison demonstrated increasing smoothing for larger windows, alongside the expected trade-off with response speed.

The main contribution of the project is a documented electronics and data-analysis workflow that connects microcontroller interfacing with basic sensor signal processing. The implementation is a biosensor-inspired educational prototype, not a glucose detector. Independent biochemical sensing, physical validation, and electrochemical characterization are necessary before any glucose-sensing performance claim could be made.

8 Portfolio Presentation

A Arduino Source Code

The following listing combines the final LCD, noise-injection, and five-sample moving-average implementation described in the report. It uses the pin mapping documented in Table 2.

Listing 1: Arduino program for the simulated sensor interface.

```
#include <LiquidCrystal.h>

// LCD: RS, E, D4, D5, D6, D7
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

const int sensorPin = A0;

// Illustrative calibration parameters
const float V_MIN = 0.5;
const float V_MAX = 4.5;
const float C_MIN = 0.0;
const float C_MAX = 10.0;

// Five-sample moving-average filter
const int FILTER_SIZE = 5;
int readings[FILTER_SIZE];
long readingSum = 0;
int readIndex = 0;

void setup() {
  lcd.begin(16, 2);
  Serial.begin(9600);
```

```
23
24    // Simulation-oriented pseudo-random seed.
25    // A floating A1 pin is not a robust hardware entropy source.
26    randomSeed(analogRead(A1));
27
28    // Initialize the moving-average buffer
29    for (int i = 0; i < FILTER_SIZE; i++) {
30      readings[i] = analogRead(sensorPin);
31      readingSum += readings[i];
32    }
33
34    lcd.setCursor(0, 0);
35    lcd.print("Glucose Sensor");
36    lcd.setCursor(0, 1);
37    lcd.print("Initializing...");
38    delay(2000);
39    lcd.clear();
40
41    Serial.println(
42      "Noisy_ADC,Filtered_ADC,Voltage_V,Glucose_mM"
43    );
44  }
45
46  void loop() {
47    // Read the potentiometer-emulated sensor voltage
48    int sensorADC = analogRead(sensorPin);
49
50    // Inject an integer disturbance from -20 through +20 counts
51    int noise = random(-20, 21);
52    int rawADC = constrain(sensorADC + noise, 0, 1023);
53
54    // Update the circular buffer and running sum
55    readingSum -= readings[readIndex];
56    readings[readIndex] = rawADC;
57    readingSum += rawADC;
58
59    readIndex++;
60    if (readIndex >= FILTER_SIZE) {
61      readIndex = 0;
62    }
63
64    // Moving-average output
65    float filteredADC =
66      (float)readingSum / FILTER_SIZE;
67
68    // Convert filtered ADC code to nominal voltage
69    float voltage = filteredADC * (5.0 / 1023.0);
70
71    // Convert voltage to glucose-equivalent concentration
72    float glucose = (voltage - V_MIN) *
73                    (C_MAX - C_MIN) /
74                    (V_MAX - V_MIN);
75    glucose = constrain(glucose, C_MIN, C_MAX);
76
77    // Update LCD
78    lcd.setCursor(0, 0);
79    lcd.print("Sensor: ");
80    lcd.print(voltage, 2);
81    lcd.print("V   ");
82
```

```
  lcd.setCursor(0, 1);
  lcd.print("Glucose ");
  lcd.print(glucose, 2);
  lcd.print("mM ");

  // CSV output for Python analysis
  Serial.print(rawADC);
  Serial.print(",");
  Serial.print(filteredADC, 2);
  Serial.print(",");
  Serial.print(voltage, 3);
  Serial.print(",");
  Serial.println(glucose, 3);

  delay(200);
}
```

Implementation note: The Arduino code above includes the software noise injection used in the later filtering test. To run a clean potentiometer-to-LCD demonstration without injected noise, replace the two lines that generate `noise` and `rawADC` with `int rawADC = sensorADC;`. The filter will then smooth ADC variation that is present in the input, but a stable simulator input may show little fluctuation.

B Python Analysis Code

B.1 Calibration and regression

Listing 2: Calibration dataset and least-squares fit.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Reported calibration observations
calibration = pd.DataFrame({
    "Voltage_V": [0.499, 1.500, 2.498, 3.500, 4.501],
    "Glucose_equiv_mM": [0.000, 2.501, 4.994, 7.499, 10.000]
})

x = calibration["Voltage_V"].to_numpy()
y = calibration["Glucose_equiv_mM"].to_numpy()

slope, intercept = np.polyfit(x, y, 1)
y_fit = slope * x + intercept

ss_res = np.sum((y - y_fit) ** 2)
ss_tot = np.sum((y - np.mean(y)) ** 2)
r_squared = 1 - ss_res / ss_tot

print(f"Slope: {slope:.4f} mM/V")
print(f"Intercept: {intercept:.4f} mM")
print(f"R-squared: {r_squared:.6f}")

plt.figure(figsize=(7, 5))
plt.scatter(x, y, label="Reported simulated points")
order = np.argsort(x)
plt.plot(x[order], y_fit[order], label="Least-squares fit")
plt.xlabel("Sensor output voltage (V)")
```

```
plt.ylabel("Glucose-equivalent concentration (mM)")
plt.title("Illustrative calibration mapping")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```

B.2 Gaussian-noise demonstration

Listing 3: Separate Python demonstration using Gaussian noise.

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)
n_samples = 200
true_voltage = 2.5
noise_std = 0.05

noise = np.random.normal(0, noise_std, n_samples)
raw_voltage = np.clip(true_voltage + noise, 0, 5)

print(f"Mean voltage: {np.mean(raw_voltage):.4f} V")
print(f"Voltage standard deviation: {np.std(raw_voltage):.4f} V")

plt.figure(figsize=(9, 4))
plt.plot(raw_voltage, label="Noisy signal")
plt.axhline(true_voltage, linestyle="--", label="Ideal signal")
plt.xlabel("Sample number")
plt.ylabel("Voltage (V)")
plt.title("Python-generated Gaussian-noise example")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```

B.3 Arduino CSV analysis and window comparison

Listing 4: Load an Arduino CSV capture and compare common-window moving averages.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load the CSV exported/copied from the Arduino Serial Monitor.
# Expected columns:
# Noisy_ADC,Filtered_ADC,Voltage_V,Glucose_mM
measured_df = pd.read_csv("data/arduino_noise_capture.csv")
raw_signal = measured_df["Noisy_ADC"].astype(float)

window_sizes = [3, 5, 9]
trim = max(window_sizes) // 2
raw_common = raw_signal.iloc[trim:-trim]

results = []
plt.figure(figsize=(10, 5))
plt.plot(raw_common.index, raw_common,
         label="Raw ADC", alpha=0.5)
```

```

for window in window_sizes:
    filtered = (
        raw_signal
        .rolling(window=window, center=True)
        .mean()
        .iloc[trim:-trim]
    )

    raw_std = raw_common.std(ddof=0)
    filtered_std = filtered.std(ddof=0)
    reduction = (1 - filtered_std / raw_std) * 100

    results.append({
        "Window Size": window,
        "Filtered Std": filtered_std,
        "Reduction (%)": reduction
    })

    plt.plot(filtered.index, filtered,
             label=f"Window = {window}")

plt.xlabel("Sample number")
plt.ylabel("ADC reading")
plt.title("Moving-average filter comparison")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()

results_df = pd.DataFrame(results)
print(results_df.round(3))
```

C References

References

[1] Arduino, “Analog Input,” Arduino Built-in Examples. Available: https://docs.arduino.cc/built-in-examples/analog/AnalogInput/. Accessed: 1 November 2026.

[2] Arduino, “Arduino API / Language Reference,” including analog I/O and random-number functions. Available: https://docs.arduino.cc/learn/programming/reference. Accessed: 1 November 2026.

[3] Arduino, “LiquidCrystal Library,” Arduino Documentation. Available: https://docs.arduino.cc/libraries/liquidcrystal/. Accessed: 1 November 2026.

[4] Autodesk, “Tinkercad Circuits.” Available: https://www.tinkercad.com/circuits. Used as the circuit simulation environment.